

```

#include <SPI.h>
#include <Ethernet.h>

/* ===== FIRMWARE VERSION ===== */
#define FW_VERSION "2.0.0"

/* ===== W5500 PIN CONFIG ===== */
#define W5500_CS 5
#define W5500_RST 4
#define SPI_SCK 18
#define SPI_MISO 19
#define SPI_MOSI 23

/* ===== ETHERNET CONFIG ===== */
byte mac[] = {0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED};
IPAddress ip(192, 168, 1, 180);
IPAddress dns(8, 8, 8, 8);
IPAddress gateway(192, 168, 1, 1);
IPAddress subnet(255, 255, 255, 0);

/* ===== MEGA CONFIG ===== */
const char* megaIP = "192.168.1.178";
const int megaPort = 9000;
// v2.0.0: timeout chờ MEGA phản hồi – MEGA có thể delay tối đa ~120ms (readIO x2)
#define MEGA_TIMEOUT_MS 300

/* ===== LOGO! MODBUS TCP CONFIG ===== */
const char* logoIP = "192.168.1.6";
const int logoPort = 502;
#define LOGO_UNIT_ID 1
#define LOGO_M_ADDR 4
#define LOGO_FB_ADDR 0
#define LOGO_PULSE_MS 500
#define LOGO_COOLDOWN_MS 1500
#define LOGO_MODBUS_TIMEOUT_MS 200

/* ===== SERVER ===== */
EthernetServer server(80);
// v2.0.0: KHÔNG có persistent megaClient – dùng connect-per-transaction
// Đây là giải pháp dứt điểm cho vấn đề "RECONNECTED OK" loop dưới tải cao:
// persistent connection bị drop bởi W5500 khi nhiều HTTP socket mở/đóng đồng thời.
// Mỗi lệnh relay: connect → gửi → đọc response → disconnect. Không bao giờ stale.
String globalStatus = "";
uint8_t acSnapshot = 0;

/* ===== LOGO! PULSE STATE MACHINE ===== */
struct LogoPulse { bool active; int channel; unsigned long startMs; };
LogoPulse logoPulse = {false, -1, 0};
unsigned long acCooldownMs[4] = {0, 0, 0, 0};

```

```

String normalizeStatusLine(const String &raw) {
    int start = raw.lastIndexOf("L1=");
    if (start < 0) return "";
    String part = raw.substring(start);
    part.replace("\r", ""); part.replace("\n", "");
    int end = part.lastIndexOf("L16=");
    if (end < 0) return part;
    int commaAfter = part.indexOf(',', end);
    if (commaAfter > 0) return part.substring(0, commaAfter);
    return part;
}

/* ===== RESET W5500 ===== */
void resetW5500() {
    pinMode(W5500_RST, OUTPUT);
    digitalWrite(W5500_RST, HIGH); delay(100);
    digitalWrite(W5500_RST, LOW); delay(200);
    digitalWrite(W5500_RST, HIGH); delay(200);
}

/* ===== MEGA TRANSACT (v2.0.0 – connect-per-transaction) ===== */
// Gửi 1 lệnh tới MEGA, đọc response, cập nhật globalStatus, đóng kết nối.
// Pattern giống LOGO! logoTransact() – không giữ persistent connection → không bao
giờ stale/drop.
// Blocking tối đa MEGA_TIMEOUT_MS (300ms) – chấp nhận được vì MEGA thường phản hồi
<120ms.
bool megaTransact(const String &cmd) {
    EthernetClient c;
    if (!c.connect(megaIP, megaPort)) {
        Serial.println("MEGA connect fail");
        return false;
    }
    Serial.println("TX: " + cmd);
    c.print(cmd + "\r\n");

    // Đọc response line từ MEGA
    unsigned long t0 = millis();
    String line = "";
    while (millis() - t0 < MEGA_TIMEOUT_MS) {
        while (c.available()) {
            char ch = (char)c.read();
            if (ch == '\n') {
                line.trim();
                String normalized = normalizeStatusLine(line);
                if (normalized.length() > 0) globalStatus = normalized;
                c.stop();
                return true;
            } else if (ch != '\r') {
                line += ch;
            }
        }
    }
}

```

```

    }
    }
    yield();
}
// Timeout – cập nhật status nếu có gì
line.trim();
String normalized = normalizeStatusLine(line);
if (normalized.length() > 0) globalStatus = normalized;
c.stop();
return false;
}

/* ===== MODBUS TCP – LOGO! (connect-per-transaction) ===== */
bool logoTransact(byte* reqFrame, int reqLen, byte* respBuf, int expectedLen) {
    EthernetClient c;
    if (!c.connect(logoIP, logoPort)) return false;
    c.write(reqFrame, reqLen);
    unsigned long t0 = millis();
    while (c.available() < expectedLen && millis() - t0 < LOGO_MODBUS_TIMEOUT_MS) {
yield(); }
    bool ok = (c.available() >= expectedLen);
    if (ok && respBuf) c.readBytes(respBuf, expectedLen);
    while (c.available()) c.read();
    c.stop();
    return ok;
}

void logoFC5(uint16_t addr, bool val) {
    byte frame[12] = {0x00,0x01,0x00,0x00,0x00,0x06,LOGO_UNIT_ID,0x05,
        (byte)(addr>>8),(byte)(addr&0xFF),(byte)(val?0xFF:0x00),0x00};
    logoTransact(frame, 12, nullptr, 12);
}

void logoSendPulse(int ch) {
    if (logoPulse.active) return;
    if (millis() - acCooldownMs[ch] < LOGO_COOLDOWN_MS) return;
    logoFC5(LOGO_M_ADDR + ch, true);
    logoPulse = {true, ch, millis()};
    acCooldownMs[ch] = millis();
}

/* ===== HTTP HANDLER ===== */
void handleWebRequest(EthernetClient client) {
    String request = "";
    bool headerComplete = false;
    unsigned long t0 = millis();
    while (!headerComplete && millis() - t0 < 30) {
        while (client.available()) {
            char c = client.read();
            request += c;

```

```

        if (request.indexOf("\r\n\r\n") >= 0) { headerComplete = true; break; }
    }
    if (headerComplete || !client.connected()) break;
    yield();
}
if (!headerComplete) { client.stop(); return; }

// ===== API SET =====
if (request.indexOf("GET /set?") >= 0) {
    int rPos = request.indexOf("relay=") + 6;
    int vPos = request.indexOf("value=") + 6;
    int rEnd = request.indexOf("&", rPos);
    int vEnd = request.indexOf(" ", vPos);
    if (rEnd == -1) rEnd = request.indexOf(" ", rPos);
    String relayNum = request.substring(rPos, rEnd);
    String valStr = request.substring(vPos, vEnd);
    bool isOn = (valStr == "true" || valStr == "1");
    megaTransact("set /relay/" + relayNum + "/state " + (isOn ? "true" : "false"));
    client.println("HTTP/1.1 200 OK\r\nContent-Type:
application/json\r\nConnection: close\r\n");
    client.print("{\"status\":\"success\",\"relay\":"); client.print(relayNum);
    client.print(",\"value\":"); client.print(isOn ? "true" : "false");
client.println("}");
    client.stop(); return;
}

// ===== API STATUS JSON =====
if (request.indexOf("GET /api/status") >= 0) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type:
application/json\r\nConnection: close\r\n");
    client.print("{\"raw\":\""); client.print(globalStatus); client.println("\"}");
    client.stop(); return;
}

/* ===== WEB UI MENU ===== */
if (request.startsWith("GET / ")) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: text/html;
charset=UTF-8\r\nConnection: close\r\n");
    client.println("<!DOCTYPE html><html><head><meta charset='UTF-8'><meta
name='viewport' content='width=device-width, initial-scale=1'>");

client.println("<style>body{font-family:Arial;background:#111;color:#fff;text-align
:center;padding:20px;}h2{margin-top:10px}.menu{max-width:500px;margin:30px
auto;display:grid;gap:14px;}a{display:block;padding:18px;border-radius:12px;backgro
und:#333;color:#fff;text-decoration:none;font-size:18px;border:1px solid
#555;}a:hover{background:#444;}</style></head><body>");
    client.println("<h2>TNN SI - SMART CONTROL</h2><p>Chọn nhóm thiết bị cần điều
khiển:</p>");
    client.println("<div class='menu'>");
    client.println("<a href='/mega'>🕹️ Điều khiển Đèn (MEGA)</a>");

```

```

    client.println("<a href='/logo'>⚙️ Điều khiển Điều hòa (LOGO!)</a>");
    client.println("<a href='/amx'>⚙️ Điều khiển thiết bị AMX</a>");
    client.println("<a href='/modbus'>⚙️ Điều khiển Modbus TCP</a>");
    client.println("</div></body></html>");
    client.stop(); return;
}

/* ===== WEB UI MEGA ===== */
if (request.startsWith("GET /mega ")) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: text/html;
charset=UTF-8\r\nConnection: close\r\n");
    client.println("<!DOCTYPE html><html><head><meta charset='UTF-8'><meta
name='viewport' content='width=device-width, initial-scale=1'>");

    client.println("<style>body{font-family:Arial;background:#111;color:#fff;text-align
:center;}");

    client.println(".top{display:flex;justify-content:center;gap:10px;align-items:cente
r;flex-wrap:wrap;}");
    client.println(".back{display:inline-block;padding:8px
12px;border-radius:8px;background:#333;color:#fff;text-decoration:none;}");

    client.println(".grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(100
px,1fr));gap:10px;}");

    client.println("button{height:60px;font-size:12px;border-radius:10px;border:none;ba
ckground:#444;color:#fff;transition:0.1s;cursor:pointer;}");
    client.println("button:active{transform:scale(0.95);background:#666;}");
    client.println("button.pressing{transform:scale(0.95);background:#888;}");
    client.println(".on{background:green;color:#fff;}");
    client.println("button.on{background:green;}");
    client.println("</style></head><body>");
    client.println("<div class='top'><a class='back' href='/'>⬅️ Menu</a><h2>TNN SI
- MEGA CONTROL</h2></div>");
    client.println("<h4>RELAY CONTROL (1-16)</h4><div class='grid'>");
    const char* names[] = {"Kho","H.Lang","P.Hợp","Test
Đèn","Lab","Còi","K.Doanh","N.Anh","P.Nguyên","P.Tuấn","Bàn Trà","Kế
Toán","L13","L14","L15","L16"};
    for (int i = 1; i <= 16; i++) {
        client.print("<button id='L"); client.print(i); client.print("'>");
        client.print(i); client.print(". "); client.print(names[i-1]);
    client.println("</button>");
    }
    client.println("<button id='ALL' style='background:#aa0000;'>🛑 TẮT TẮT
CẢ</button></div>");
    client.println("<pre id='status'
style='font-size:10px;margin-top:20px;opacity:0.5;'>Loading...</pre>");
    client.println("<script>");
    client.println("let lastCmdTime=0;let cmdInFlight=false;let
queuedCmd=null;const CMD_COOLDOWN_MS=150;");

```

```

        client.println("let pendingTarget={};");
        client.println("function
applyStatus(t){document.getElementById('status').innerText=t;t.split(',').forEach(p
=>{let kv=p.split('=');if(kv.length!=2)return;let k=kv[0].trim();let
v=kv[1].trim();let b=document.getElementById(k);if(!b)return;let
isOn=(v=='1');if(isOn)b.classList.add('on');else
b.classList.remove('on');pendingTarget[k]=isOn;}});");
        client.println("function sendCmd(c,btn){let
now=Date.now();if(now-lastCmdTime<CMD_COOLDOWN_MS){queuedCmd={c,btn};setTimeout(()=
>{if(!cmdInFlight&&queuedCmd){const
q=queuedCmd;queuedCmd=null;sendCmd(q.c,q.btn);}},CMD_COOLDOWN_MS);return;}lastCmdTi
me=now;cmdInFlight=true;if(btn){btn.classList.add('pressing');setTimeout(()=>btn.cl
assList.remove('pressing'),90);}fetch('/cmd?c='+encodeURIComponent(c)).then(()=>set
Timeout(poll,80)).catch(()=>setTimeout(poll,150)).finally(()=>{cmdInFlight=false;if
(queuedCmd){const q=queuedCmd;queuedCmd=null;sendCmd(q.c,q.btn);}});});");
        client.println("function
cmd(c,btn){if(cmdInFlight){queuedCmd={c,btn};return;}sendCmd(c,btn);});");
        client.println("for(let i=1;i<=16;i++){let b=document.getElementById('L'+i);let
key='L'+i;if(b)b.onclick=(()=>{let current=(key in
pendingTarget)?pendingTarget[key]:b.classList.contains('on');let
next=!current;pendingTarget[key]=next;b.classList.toggle('on',next);cmd('set
/relay/'+i+'/state '+next?'true':'false'),b);}});");

client.println("document.getElementById('ALL').onclick=(e)=>{pendingTarget={};cmd('
set /system/all off',e.target);});");
        client.println("function
poll(){fetch('/status').then(r=>r.text()).then(t=>applyStatus(t));});");
        client.println("setInterval(poll,2000);poll();");
        client.println("</script></body></html>");
        client.stop(); return;
    }

    /* ===== WEB UI LOGO! ===== */
    if (request.startsWith("GET /logo ")) {
        client.println("HTTP/1.1 200 OK\r\nContent-Type: text/html;
charset=UTF-8\r\nConnection: close\r\n");
        client.println("<!DOCTYPE html><html><head><meta charset='UTF-8'><meta
name='viewport' content='width=device-width, initial-scale=1'>");

client.println("<style>*{margin:0;padding:0;box-sizing:border-box;}body{display: fle
x;flex-direction:column;height:100vh;background:#111;font-family:Arial;}.bar{displa
y:flex;align-items:center;gap:8px;padding:7px
10px;background:#1a1a1a;border-bottom:1px solid
#333;flex-shrink:0;flex-wrap:wrap;}.back{padding:6px
10px;border-radius:7px;background:#333;color:#fff;text-decoration:none;font-size:13
px;white-space:nowrap;}.back:hover{background:#444;}.title{color:#fff;font-size:14p
x;font-weight:bold;flex:1;}.btn{padding:6px
10px;border-radius:7px;color:#fff;text-decoration:none;font-size:13px;white-space:n
owrap;cursor:pointer;border:none;}.btn-blue{background:#225588;}.btn-green{backgrou
nd:#226633;}.hint{background:#2a2000;border-bottom:1px solid #554400;padding:8px

```

```

12px;font-size:12px;color:#ffcc66;flex-shrink:0;}.hint
b{color:#ffee99;}iframe{flex:1;width:100%;border:none;}</style></head><body>");
    client.println("<div class='bar'><a class='back' href='/'>← Menu</a><span
class='title'>⊗ Điều hòa (LOGO!)</span>");
    client.println("<a class='btn btn-blue'
href='http://192.168.1.6/webroot/main.htm' target='_blank'>↗ Mở mới</a>");
    client.println("<button class='btn btn-green'
onclick='document.getElementById(\"lf\").src=document.getElementById(\"lf\").src'>↶
Tải lại</button></div>");
    client.println("<div class='hint'>⚠ Nếu thấy trang đăng nhập: nhấn <b>↗ Mở
mới</b> → đăng nhập → quay lại → nhấn <b>↶ Tải lại</b></div>");
    client.println("<iframe id='lf'
src='http://192.168.1.6/webroot/main.htm'></iframe>");
    client.println("</body></html>");
    client.stop(); return;
}

/* ===== WEB UI AMX ===== */
if (request.startsWith("GET /amx ")) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: text/html;
charset=UTF-8\r\nConnection: close\r\n");
    client.println("<!DOCTYPE html><html><head><meta charset='UTF-8'><meta
name='viewport' content='width=device-width, initial-scale=1'>");

    client.println("<style>body{font-family:Arial;background:#111;color:#fff;text-align
:center;padding:16px;}.top{display:flex;justify-content:center;gap:10px;align-items
:center;flex-wrap:wrap;}.back{display:inline-block;padding:8px
12px;border-radius:8px;background:#333;color:#fff;text-decoration:none;}.grid{displ
ay:grid;grid-template-columns:repeat(2,minmax(120px,1fr));gap:10px;max-width:500px;
margin:20px
auto;}button{height:70px;font-size:14px;border-radius:10px;border:none;background:#
225588;color:#fff;}small{opacity:.75;display:block;margin-top:8px;}</style></head><
body>");
    client.println("<div class='top'><a class='back' href='/'>← Menu</a><h2>AMX
CONTROL</h2></div>");
    client.println("<p>Trang giao diện AMX (khung cơ bản 4 nút, sẽ gắn lệnh
sau).</p>");
    client.println("<div class='grid'><button>AMX 1</button><button>AMX
2</button><button>AMX 3</button><button>AMX 4</button></div>");
    client.println("<small>Hiện tại các nút đang ở chế độ
placeholder.</small></body></html>");
    client.stop(); return;
}

/* ===== WEB UI MODBUS TCP (placeholder) ===== */
if (request.startsWith("GET /modbus ")) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: text/html;
charset=UTF-8\r\nConnection: close\r\n");
    client.println("<!DOCTYPE html><html><head><meta charset='UTF-8'><meta
name='viewport' content='width=device-width, initial-scale=1'>");

```

```

client.println("<style>body{font-family:Arial;background:#111;color:#fff;text-align
:center;padding:20px;}.top{display:flex;justify-content:center;gap:10px;align-items
:center;flex-wrap:wrap;}.back{display:inline-block;padding:8px
12px;border-radius:8px;background:#333;color:#fff;text-decoration:none;}.info{max-w
idth:480px;margin:40px auto;background:#1a1a1a;border:1px solid
#444;border-radius:12px;padding:24px;}.badge{display:inline-block;background:#55330
0;color:#ffaa55;border-radius:6px;padding:4px
10px;font-size:12px;margin-bottom:12px;}.desc{color:#aaa;font-size:14px;line-height
:1.6;}</style></head><body>");
    client.println("<div class='top'><a class='back' href='/'>← Menu</a><h2>⚙
MODBUS TCP CONTROL</h2></div>");
    client.println("<div class='info'><span class='badge'>ĐANG PHÁT TRIỂN</span>");
    client.println("<p class='desc'>Trang điều khiển thiết bị qua Modbus TCP trực
tiếp.<br><br>Chưa hoạt động, sẽ hoàn thiện trong phiên bản tiếp theo.</p></div>");
    client.println("</body></html>");
    client.stop(); return;
}

/* ===== CMD ===== */
if (request.indexOf("GET /cmd?") >= 0) {
    int s = request.indexOf("c=") + 2;
    int e = request.indexOf(" ", s);
    String cmd = request.substring(s, e);
    cmd.replace("%20", " "); cmd.replace("%2F", "/");
    megaTransact(cmd); // connect → send → read status → disconnect
    client.println("HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\nConnection:
close\r\n");
    client.println("OK");
    client.stop(); return;
}

if (request.indexOf("GET /status") >= 0) {
    // Trả cached globalStatus – được cập nhật bởi megaTransact() sau mỗi lệnh
    client.println("HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\n" +
globalStatus);
    client.stop(); return;
}

client.stop();
}

/* ===== SETUP ===== */
void setup() {
    Serial.begin(115200);
    Serial.print("ESP32 Firmware v"); Serial.println(FW_VERSION);

    resetW5500();
    SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, W5500_CS);
    SPI.setFrequency(8000000);

```



```

Ethernet.init(W5500_CS);
Ethernet.begin(mac, ip, dns, gateway, subnet);
delay(1000);

Serial.print("IP: "); Serial.println(Ethernet.localIP());
server.begin();

// Đọc trạng thái ban đầu từ MEGA
megaTransact("get /relay/all");
}

/* ===== LOOP ===== */
void loop() {

    EthernetClient client = server.available();
    if (client) handleWebRequest(client);

    // LOGO! pulse completion – non-blocking
    if (logoPulse.active && millis() - logoPulse.startMs >= LOGO_PULSE_MS) {
        logoFC5(LOGO_M_ADDR + logoPulse.channel, false);
        logoPulse.active = false;
    }

    // Sync trạng thái relay định kỳ (phòng trường hợp thay đổi từ công tắc vật lý /
scene)
    static unsigned long lastSync = 0;
    if (millis() - lastSync >= 3000) {
        lastSync = millis();
        megaTransact("get /relay/all");
    }
}

```